

SIMATS ENGINEERING

(SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2

CI/CD Pipeline Design Exercise

Course Code	CSA1017	Course Title	Software Engineering
CO Assessed	CO4	Assessment	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage	20%	Total Marks	25

CO4 Statement: Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics.

Student Name	RR Thiyagesh	Reg. No.	192572007
Date	August 2026	Duration	60 minutes

Assigned Problem Statement

Intelligent AI-Based Airport Baggage Tracking and Management System

An AI-powered platform that monitors baggage movement using RFID & IoT, optimizes routing, predicts delays, notifies passengers, and generates operational analytics to improve baggage handling efficiency and reduce passenger complaints.

Code of Conduct

I, **RR Thiyagesh (Reg. No: 192572007)**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Deliverable: CI/CD Pipeline Design document containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

1. Requirement Analysis & CI/CD Needs Identification

1.1 System Overview

The Intelligent AI-Based Airport Baggage Tracking and Management System is a cloud-native, microservices-based platform that integrates RFID readers, IoT sensors, AI/ML models, and mobile/web interfaces to provide real-time baggage tracking, predictive delay detection, automated passenger notifications, optimized routing, and operational analytics. The system must handle high-volume real-time data streams from multiple airport terminals while ensuring low latency, high availability, data privacy, and regulatory compliance.

1.2 Key Functional Components

- **RFID & IoT Ingestion Service:** Collects and normalizes tag scan events from baggage handling systems.
- **Real-time Tracking Engine:** Maintains current location and status of every bag using event streaming.
- **AI Delay Prediction Module:** Uses historical and live data (weather, flight delays, conveyor load) to forecast misrouting or late delivery.
- **Routing Optimization Service:** Recommends optimal conveyor/gate paths using constraint-based algorithms.
- **Notification Service:** Pushes status updates and alerts to passengers via mobile app, SMS, and email.
- **Analytics & Reporting Dashboard:** Generates operational KPIs, loss rates, and performance reports for airport staff.
- **Admin & Configuration Portal:** Allows staff to manage rules, thresholds, and system parameters.

1.3 Identified CI/CD Requirements

Given the mission-critical nature of airport operations, the CI/CD pipeline must satisfy the following needs:

Need	Description	Priority
Frequent, safe releases	Multiple microservices and ML models must be updated without downtime.	Critical
Automated testing	Unit, integration, API, load, and ML model validation must run on every change.	Critical
Security & compliance	Container scanning, SAST/DAST, secrets management, and audit trails required.	Critical
Environment parity	Dev, Staging, and Production must be as similar as possible (containers + IaC).	High
Fast feedback	Developers need results within minutes of committing code.	High
Rollback capability	Instant rollback for any failed deployment (blue-green / canary).	Critical
ML model lifecycle	Model training, validation, versioning, and safe promotion to production.	High
Observability	Automated monitoring, alerting, and logging integrated into the pipeline.	High
Auditability	Full traceability from commit to production deployment for compliance.	Medium

2. CI/CD Pipeline Architecture & Workflow

2.1 High-Level Pipeline Design

The proposed CI/CD pipeline follows a continuous integration and continuous delivery model tailored for a multi-service, AI-enabled airport system. It is triggered on every push to feature branches and on merges to the main branch. The pipeline consists of six major stages that ensure code quality, security, and reliable deployment across environments.

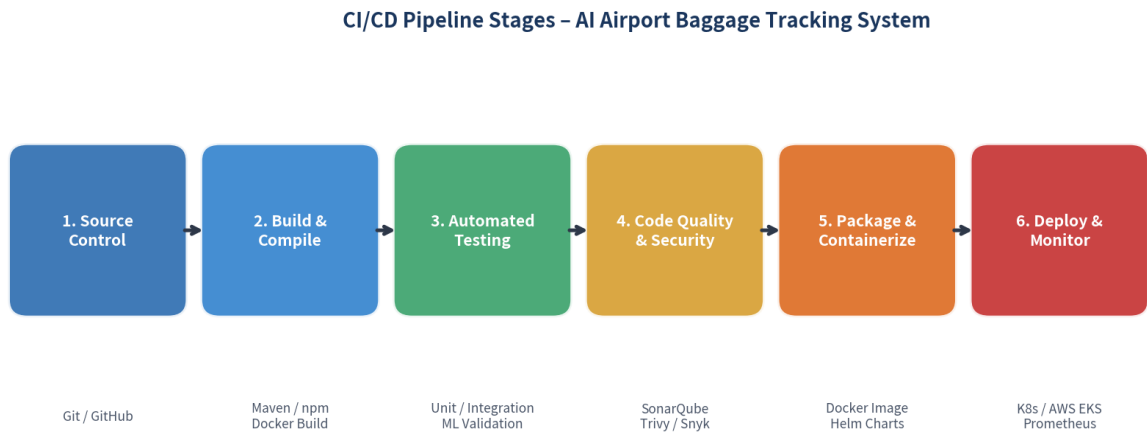


Figure 1: High-Level CI/CD Pipeline Stages

2.2 Detailed Stage Descriptions

Stage 1 – Source Code Management: All source code (backend microservices, frontend dashboards, ML training pipelines, infrastructure-as-code) resides in a GitHub monorepo or multi-repo setup. Branch protection rules enforce pull-request reviews, status checks, and signed commits. Feature branches trigger the full pipeline; main branch deployments require additional approvals.

Stage 2 – Build & Compile: Backend services (Java/Spring Boot or Node.js) are compiled and packaged. Frontend (React/Angular) is built and optimized. ML model training jobs are executed (or validated against pre-trained artifacts). Docker images are built for each service using multi-stage Dockerfiles to keep image size minimal.

Stage 3 – Automated Testing: Unit tests, integration tests, API contract tests, and component tests run in parallel. For the AI module, model evaluation metrics (accuracy, precision, recall, latency) are checked against defined thresholds. Synthetic RFID event streams are used to validate the tracking engine.

Stage 4 – Code Quality & Security Analysis: Static code analysis (SonarQube) enforces quality gates on code smells, bugs, and coverage. Dependency vulnerability scanning (Snyk/Trivy) and container image scanning are performed. Secrets detection prevents accidental credential commits.

Stage 5 – Packaging: Successful builds produce versioned Docker images pushed to a private container registry (Amazon ECR / GitHub Container Registry). Helm charts or Kubernetes manifests are versioned and stored as artifacts.

Stage 6 – Deployment & Monitoring: Images are deployed first to the Development environment automatically. After successful smoke tests, promotion to Staging requires manual approval. Production uses canary or blue-green deployment with automated health checks and instant rollback capability.

3. Tools & Technology Selection with Justification

Tool selection prioritizes industry-standard, cloud-native, open-source (or widely adopted commercial) solutions that support high availability, security, and scalability required by airport operations.

Stage	Tool / Technology	Justification
Source Control	Git + GitHub	Industry standard VCS; strong branch protection, PR reviews, Actions integration, and audit logs.
CI Orchestration	GitHub Actions	Native GitHub integration, matrix builds, reusable workflows, free for public/private repos, excellent secret management.
Build	Maven / Gradle / npm + Docker	Standard build tools for Java/Node services; Docker ensures reproducible, portable artifacts.
Unit / Integration Testing	JUnit / pytest / Jest + Testcontainers	Language-native frameworks; Testcontainers provide realistic DB/message-broker environments.
API / Contract Testing	Postman / Newman + Pact	Automated API regression; consumer-driven contracts prevent breaking changes between microservices.
ML Model Validation	MLflow + custom evaluation scripts	Tracks experiments, metrics, and model versions; enforces acceptance criteria before promotion.
Code Quality	SonarQube	Comprehensive static analysis, quality gates, technical debt tracking, and coverage reports.
Security Scanning	Trivy + Snyk + OWASP ZAP	Container & dependency vulnerability scanning; DAST for runtime security testing.
Container Registry	Amazon ECR / GHCR	Secure, private, highly available image storage with vulnerability scanning integration.
Orchestration	Kubernetes (AWS EKS)	Industry standard for container orchestration; supports rolling updates, canary, and auto-scaling.
IaC	Terraform + Helm	Declarative infrastructure; version-controlled environment parity across Dev/Staging/Prod.
Monitoring & Alerting	Prometheus + Grafana + PagerDuty	Real-time metrics, dashboards, and on-call alerting critical for 24×7 airport operations.
Notifications	Slack + Email (SMTP)	Immediate developer feedback on pipeline status and production incidents.

Rationale for Cloud Choice (AWS): Airports and airlines commonly operate on major cloud providers. AWS EKS, ECR, CloudWatch, and IAM provide a mature, secure foundation with strong compliance certifications (SOC2, ISO 27001) required for aviation data handling.

4. Automated Testing Strategy

A multi-layered testing strategy is integrated directly into the pipeline to ensure reliability of both conventional software components and AI/ML models. Tests are executed automatically on every commit and must pass quality gates before any deployment proceeds.

4.1 Testing Pyramid for the Baggage System

Layer	Scope	Tools	Pipeline Stage
Unit Tests	Individual functions, classes, ML feature transformers	JUnit, pytest, Jest	Build / Test
Integration Tests	Service + DB + message broker interactions	Testcontainers, Spring Boot Test	Test
API / Contract Tests	REST/gRPC endpoints, inter-service contracts	Newman, Pact	Test
ML Model Validation	Accuracy, latency, data drift checks	MLflow, custom scripts	Test / Quality
End-to-End / UI	Critical user journeys (passenger app, staff dashboard)	Playwright / Cypress	Staging
Performance / Load	RFID event throughput, prediction latency under peak load	k6 / Locust	Staging
Security Tests	SAST, DAST, container CVE scan	SonarQube, Trivy, ZAP	Quality / Security
Smoke / Health	Post-deployment basic health endpoints	curl / custom probes	Deploy

4.2 AI/ML Specific Testing Considerations

Because the system relies on AI for delay prediction and routing optimization, additional validation is mandatory:

- **Model Performance Gate:** New model versions must meet or exceed baseline metrics (e.g., F1-score ≥ 0.92 , prediction latency < 200 ms).
- **Data Quality Checks:** Incoming training/inference data is validated for schema, missing values, and distribution drift.
- **Shadow Mode Deployment:** New models can run in parallel with production models without affecting live decisions until validated.
- **A/B Testing Support:** Pipeline supports gradual traffic shifting between model versions.
- **Explainability & Audit:** Model decisions that affect passenger notifications are logged for later audit.

4.3 Quality Gates

The pipeline enforces strict quality gates. Any failure stops progression:

- Unit test coverage $\geq 80\%$ for critical services.
- Zero critical or high severity vulnerabilities in container images.
- SonarQube quality gate status = PASSED.
- All integration and contract tests green.
- ML model metrics within acceptable thresholds.
- No secrets detected in the commit.

5. Deployment Strategy & Environments

A three-environment strategy ensures changes are thoroughly validated before reaching passengers and airport operations. Infrastructure is defined as code (Terraform + Helm) so that environments remain consistent and reproducible.

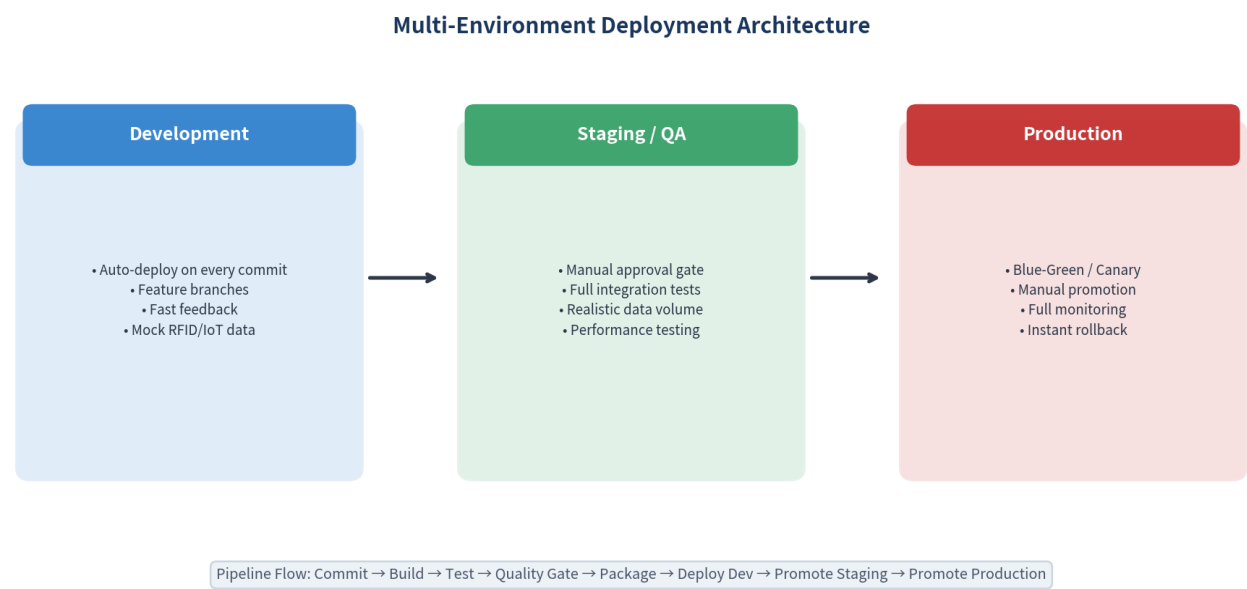


Figure 2: Multi-Environment Deployment Architecture

5.1 Environment Details

Aspect	Development	Staging / QA	Production
Trigger	Every commit / PR	Manual promotion after Dev success	Manual approval + canary
Data	Synthetic RFID events, mocks	Anonymized historical data	Live airport data streams
Scale	Minimal replicas	Production-like scale	Full auto-scaling
Purpose	Fast developer feedback	Full regression, performance, UAT	Live passenger & ops service
Access	Developers only	QA + selected stakeholders	Operations + on-call only
Rollback	Re-deploy previous image	Re-deploy previous image	Instant blue-green / canary reverse

5.2 Deployment Patterns

Blue-Green Deployment (Production): Two identical production environments (Blue = current, Green = new). Traffic is switched only after health checks and smoke tests succeed. Instant rollback by switching traffic back.

Canary Releases: New version receives a small percentage of traffic (e.g., 5% → 20% → 100%). Automated monitoring of error rates, latency, and business KPIs decides whether to proceed or roll back.

6. Security, Quality Checks, Notifications & Rollback

6.1 Security Practices Integrated in the Pipeline

- **Secrets Management:** All credentials, API keys, and certificates stored in GitHub Secrets / AWS Secrets Manager. Never committed to source control.
- **SAST (Static Application Security Testing):** SonarQube and Semgrep scan source code for common vulnerabilities (OWASP Top 10).
- **Dependency Scanning:** Snyk and Trivy check third-party libraries and base images for known CVEs on every build.
- **Container Security:** Images are scanned before push; only images signed and free of critical vulnerabilities are allowed into the registry.
- **Infrastructure Security:** Terraform plans are reviewed; security groups and IAM policies follow least-privilege principles.
- **Runtime Protection:** Network policies in Kubernetes restrict pod-to-pod communication. Service mesh (optional Istio) provides mTLS.
- **Compliance Logging:** Every pipeline run, approval, and deployment is logged for audit purposes (aviation regulatory requirements).

6.2 Notifications

Pipeline status is communicated in real time:

- Slack channel notifications on pipeline start, success, and failure (with links to logs).
- Email alerts to the commit author and on-call engineer for production deployments.
- PagerDuty integration for critical production failures or health-check degradations.
- Dashboard status badges in GitHub README and internal Confluence pages.

6.3 Rollback Mechanisms

Given the criticality of baggage operations, rollback must be fast and reliable:

- **Kubernetes Rollback:** `kubectl rollout undo` reverts to the previous ReplicaSet within seconds.
- **Blue-Green Switch:** Traffic is redirected back to the previous healthy environment instantly via load-balancer or Ingress update.
- **Canary Abort:** Automated analysis of metrics can abort a canary and restore 100% traffic to the stable version.
- **Database Migrations:** Backward-compatible migrations are preferred; a down-migration script is prepared for every schema change.
- **Model Rollback:** Previous ML model version remains available; traffic can be switched back via feature flag or model registry pointer.
- **Automated Triggers:** Elevated error rate, increased latency, or failed health checks automatically initiate rollback and notify the team.

6.4 Monitoring & Observability Post-Deployment

Prometheus scrapes metrics from all services. Grafana dashboards display baggage processing rates, prediction accuracy, notification delivery success, and infrastructure health. Alerts fire to PagerDuty when SLOs (e.g., 99.9% availability, p99 latency < 300 ms) are breached.

7. End-to-End Workflow Explanation

The following diagram and narrative describe the complete journey of a code change from a developer's commit to a live production release serving airport passengers and staff.

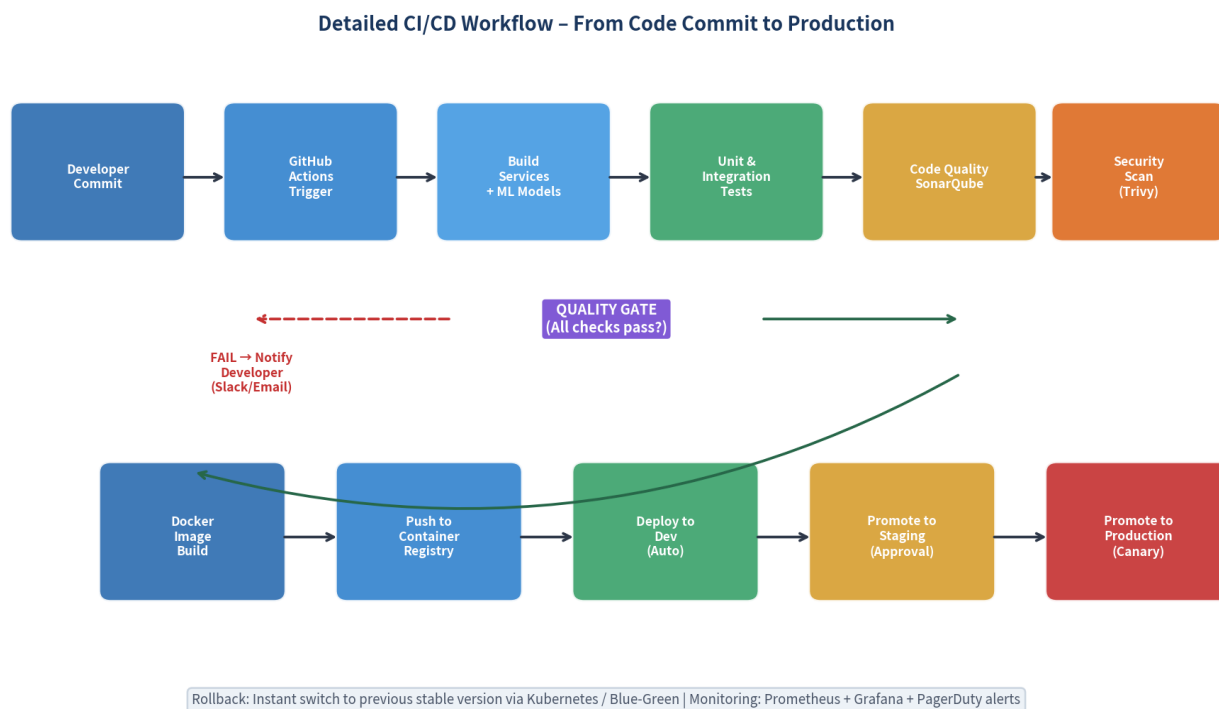


Figure 3: Detailed CI/CD Workflow – Commit to Production

7.1 Step-by-Step Workflow

- 1. Code Commit:** Developer pushes to a feature branch or opens a pull request. GitHub Actions workflow is triggered automatically.
- 2. Build:** Source is checked out. Backend services are compiled, frontend assets are built, and Docker images are constructed using multi-stage builds.
- 3. Automated Tests:** Unit, integration, and contract tests execute in parallel. ML model evaluation scripts run against a validation dataset.
- 4. Quality & Security:** SonarQube analyzes code quality. Trivy scans container images. Snyk checks dependencies. Secrets scanning runs.
- 5. Quality Gate:** If any check fails, the pipeline stops, developers are notified via Slack/Email, and the PR cannot be merged.
- 6. Package:** On success, versioned Docker images are pushed to the container registry. Helm charts are packaged as artifacts.
- 7. Deploy to Development:** Images are automatically deployed to the Dev Kubernetes namespace. Smoke tests verify basic functionality.
- 8. Promote to Staging:** After manual approval (or automatic if configured), the same artifacts are deployed to Staging. Full regression, performance, and UAT occur here.
- 9. Promote to Production:** Final approval triggers a canary or blue-green deployment. Health checks and business metrics are monitored. Full rollout follows if metrics remain healthy.
- 10. Continuous Monitoring:** Prometheus/Grafana track system health. Any anomaly can trigger an automated rollback and alert the on-call engineer.

8. Pipeline Configuration Highlights & Justification

8.1 Why This Pipeline Fits the Problem Statement

The Intelligent AI-Based Airport Baggage Tracking system has unique characteristics that the designed pipeline directly addresses:

- **Real-time & High Availability:** Canary and blue-green deployments plus instant rollback ensure that baggage tracking never experiences prolonged downtime—critical when bags are moving through the airport.
- **AI/ML Component:** Explicit model validation gates and shadow/canary support for models prevent inaccurate delay predictions from reaching passengers.
- **Multiple Stakeholders:** Separate Dev, Staging, and Production environments allow developers, QA, airport IT, and operations teams to validate changes at appropriate fidelity levels.
- **Security & Compliance:** Aviation and passenger data require strong security controls. Integrated SAST, container scanning, secrets management, and audit logging satisfy these needs.
- **Rapid Iteration:** Automated testing and fast feedback loops enable the team to improve prediction models and routing algorithms frequently without sacrificing stability.
- **Observability:** Built-in monitoring and alerting align with the objective of generating operational analytics and quickly responding to issues that could cause baggage delays or losses.

8.2 Sample Pipeline Trigger Configuration (GitHub Actions)

A typical workflow file (.github/workflows/ci-cd.yml) would include triggers on push and pull_request to main and develop branches, jobs for build, test, quality, security, package, and deploy-dev, with environment protection rules requiring reviewers for staging and production deployments. Matrix strategies allow parallel testing across services. Caching of Maven/npm dependencies and Docker layers keeps pipeline runtimes short (typically 8–15 minutes for a full successful run).

8.3 Alignment with CO4 Learning Outcomes

This design demonstrates robust testing (multi-layer automated tests), quality assurance (SonarQube gates, coverage thresholds), and DevOps practices (CI/CD, IaC, containerization, observability). Security is emphasized through scanning and secrets management; reliability is achieved via progressive delivery and rollback; ethical considerations are addressed by audit logging, data privacy controls, and transparent notification of baggage status to passengers.

9. Conclusion & Summary

This document presents a comprehensive CI/CD pipeline design for the Intelligent AI-Based Airport Baggage Tracking and Management System. The pipeline covers the full lifecycle from source control through build, multi-layer automated testing (including ML model validation), code quality and security analysis, container packaging, and progressive deployment across Development, Staging, and Production environments.

Key strengths of the design include:

- Clear separation of CI and CD stages with automated quality gates.
- Explicit support for AI/ML model lifecycle management.
- Strong security posture (SAST, container scanning, secrets management).
- Reliable deployment patterns (blue-green / canary) with instant rollback.
- Integrated monitoring, alerting, and notification mechanisms.
- Environment parity achieved through containers and Infrastructure-as-Code.
- Traceability and auditability suitable for regulated aviation environments.

By implementing this pipeline, the development team can deliver frequent, high-quality, secure updates to the baggage tracking platform, thereby reducing baggage loss and delays, improving operational efficiency, and enhancing passenger satisfaction—directly fulfilling the project objectives.

10. References & Suggested Further Reading

- Humble, J. & Farley, D. – Continuous Delivery (Addison-Wesley).
- Kim, G., Humble, J., Debois, P., Willis, J. – The DevOps Handbook.
- GitHub Actions Documentation – <https://docs.github.com/en/actions>
- Kubernetes Documentation – <https://kubernetes.io/docs/>
- OWASP DevSecOps Guideline – <https://owasp.org/>
- MLflow Documentation – <https://mlflow.org/docs/latest/index.html>
- AWS Well-Architected Framework – DevOps & CI/CD best practices.

End of Document

RR Thiyagesh | Reg. No: 192572007 | CSA1017 – Software Engineering
CO4 Assessment Tool 2 – CI/CD Pipeline Design Exercise